EFFICIENT IMPLEMENTATION OF TRIDENDRIFORM SCHROEDER TREE ALGEBRA

P. CATOIRE, J. FROMENTIN

Abstract. We introduce a primitive computation problem in the free tridendriform algebra generated by one element which is a Hopf algebra based on Schroeder trees. We know a complex way to generate all of them, hence we want to implement this method on a computer. However, we need to create some tools to implement Schroeder trees and the multiplications over this algebra to be able to compute the primitive elements. In this paper, we detail how we made the problem mathematically understandable for a computer and how we did implement it.

1. Introduction

1.1. About the paper.

Its aim. The objective of this work is to implement the objects and operations in a Hopf algebra involving Schroeder trees. Those trees introduced in definition 1 can be endowed with three linear maps $\prec, \succ$ and $\cdot$ making it a free tridendriform algebra structure [**?**] where the products can be described with quasi-shuffles (theorem 1) . Tridendriform algebras arise in different topics in mathematics like free probability [**?**], the study of Rota-Baxter algebras [**?**] or in other topics like number theory [**?**]. Some computation tools already exist for some algebras on Sagemath [**?**, **?**] but not for all. Hence, implementing the free tridendriform algebra [**?**] (over one generator) allows us to implement any tridendriform algebras coding a quotient map by a tridendriform ideal from Schroeder trees to the desired objects.

This algebra turns out to be a Hopf algebra with its coproduct given in theorem 2. Computing the space of primitive elements of this algebra, we describe at least the largest cocommutative algebra inside it thanks to the Cartier-Quillen-Milnor-Moore theorem [**?**]. An inductive algorithm is given in the article [**?**] (shown in figure 1) to compute those elements using only $\prec, \succ$ and $\cdot$ operations.

Initially, we wanted to implement efficiently (i.e with low complexity) this algorithm to compute all primitive elements in any degree. But to our surprise, using some naive approaches, this task is really difficult. Hence, we need to choose a point of view of the problem to get a clear and efficient implementation of this algorithm.

To solve this problem, we used a particular representations of Schroeder trees as initial chains (definition 9) or specific sequences over the alphabet $\{0, 1\}$ (definition 12). Those representations are called *codes* of the tree. Thanks to this point of view, we are able to describe the action of quasi-shuffles on a pair of codes in definition 19 such that it describes the initial action of these elements onto a pair of Schroeder trees (theorem 1). We present the implementation in $C++$ of the trees as codes and of the three basics operations $\prec, \succ$ and $\cdot$ on codes that are equivalent to the original ones on Schroeder trees. We evaluate its time and memory complexity.

Its structure. The paper is divided in the following way:

- Section 2 describes and gives the minimum piece of information needed to understand the general mathematical setting in which we are working. We introduce the main objects of this work, Schroeder trees in definition 1 and quasi-shuffles in definition 4. Then,

we detail the Hopf algebra structure on $(\mathbb{K}\,\mathsf{Sch}, \prec, \succ, \cdot)$ (figure 1) and we introduce the algorithm which computes its space of primitive elements.
- Section 3 is a mathematical description of the encoding used to implement our problem. We introduce the concept of Schroeder levelled trees (definition 7), initial chains and the map associating to any initial chain a 0's and 1's sequence (definition 9). Then, we choose a way to map any Schroeder tree to a levelled Schroeder tree (proposition 1) which gives a way to get a 0's and 1's sequence from any Schroeder tree. A sequence obtained by this transformation will be called a *tree code*. Finally, we present a tridendriform structure on tree codes (definition 19) that turns out to be isomorphic to the one on Schroeder trees (corollary 1).
- The last section 4 explains the implementation we made in $C++$ to solve this problem and the results it produces.

1.2. **Notations used in the article.** In this list, we put a table of contents of notations used here :

- for any $n \in \mathbb{N}^*, [\![1, n]\!]$ is the set $\{1, \dots, n\}$;
- for any set X, we denote $\mathrm{Par}(X)$ the set of its subsets. It is endowed with the refinement order $\leq_\pi$;
- for any set $X, X^\star$ denotes the set of finite words over the alphabet X;
- $(\mathsf{Sch}, \prec, \cdot, \succ)$ the graded tridendriform algebra on Schroeder tree;
- $\mathsf{Sch}(n)$ the set of Schroeder tree with n leaves;
- FSch the set of forest of Schroeder tree;
- $\vee$ is the grafting operator of many trees onto a common root;
- given a sequence $x = (x_1, \dots, x_n)$ and $i \in [\![1, n]\!]$, we denote by $x \setminus (x_i)$ the sequence x without the term x_i;
- for reasons of length some increasing sequences will be written in columns. Doing so the minimal element of the sequence is a the bottom of the column representation of the sequence.

1.3. **Acknowledgments.**

2. About the original problem

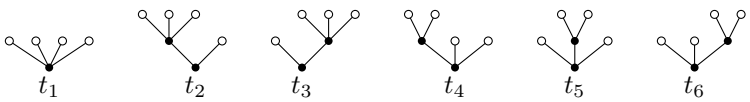
2.1. **Schroeder trees and comb representations.** Let us remind the following definition:

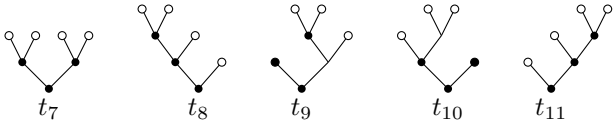
Definition 1. A *Schroeder tree* is a rooted planar tree such that any vertices which is not a leaf has at least 2 children. This set can be graded with the number of leaves minus 1. For any $n \in \mathbb{N} \setminus \{0\}$, we define $\mathsf{Sch}(n)$ the set of Schroeder trees with $n+1$ leaves. We also put:

$$\mathsf{Sch} \coloneqq \bigcup_{n=0}^{+\infty} \mathsf{Sch}(n).$$

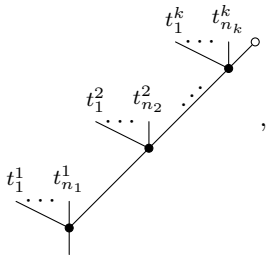
with $\mathsf{Sch}(0) = \{\mathring{|}\}$.

Example 1. The elements of $\mathsf{Sch}(4)$ are



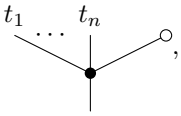


Let t be a tree. It t can always be seen as a comb:



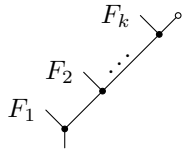
where k is the number of nodes on the right-most branch of t and for $i \in [\![1, k]\!], n_i + 1$ is the number of sons of the i-th node of this branch.

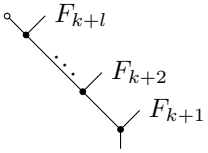
Notation 1. Let $F = t_1 \dots t_n$ be a forest composed of n trees. Instead of writing:



we will write: 

With these notations, we notice that all tree t can be seen either respectively like a *right comb* or a *left comb*:

(1) $t =$ 

and $s =$ 

,

where $F_1, \dots, F_k$ and $F_{k+1}, \dots, F_{k+l}$ are non-empty forests.

Definition 2. The writing defined above is the writing of t as a *right comb*. Proceeding the same way, looking at the left branch of the root of t, we get the writing of t as a *left comb*.

2.2. **Our objective.**

Definition 3. Let $\mathbb{K}$ be a field and A be vector space of $\mathbb{K}$. We say $(A, \prec, \cdot, \succ)$ is a *tridendriform algebra* if $\prec, \cdot$ and $\succ$ are all linear maps from $A \otimes A$ to A such that for any $a, b, c \in A$:

$$(a \prec b) \prec c = a \prec (b * c), \tag{2}$$
$$(a \succ b) \prec c = a \succ (b \prec c), \tag{3}$$
$$(a * b) \succ c = a \succ (b \succ c), \tag{4}$$
$$(a \succ b) \cdot c = a \succ (b \cdot c), \tag{5}$$
$$(a \prec b) \cdot c = a \cdot (b \succ c), \tag{6}$$
$$(a \cdot b) \prec c = a \cdot (b \prec c), \tag{7}$$
$$(a \cdot b) \cdot c = a \cdot (b \cdot c), \tag{8}$$

where $*$ is the *associative product* of the tridendriform algebra defined for any $a, b \in A$ by $a * b \coloneqq a \prec b + a \cdot b + a \succ b$. We call the products $\succ, \prec, \cdot$ respectively right, left and middle.

The free tridendriform algebra is built on the vector space $\mathbb{K}\,\mathsf{Sch} \oplus \mathbb{K}\cdot|$ where $|$ is the unit of the product $*$. We will denote it by $(\mathsf{Sch}, \prec, \cdot, \succ)$. We have a combinatorial interpretation of the product $*$. For this let us introduce:

Definition 4 (Quasi-shuffle)**.** Let $k, l \in \mathbb{N} \setminus \{0\}$. A (k,l)-*quasi-shuffle* is a surjective map $\sigma : [\![1, k+l]\!] \twoheadrightarrow [\![1, n]\!]$ surjective for some positive integer n such that:

$$\sigma(1) < \cdots < \sigma(k) \text{ and } \sigma(k+1) < \cdots < \sigma(k+l).$$

We will denote $\mathrm{QSh}(k,l)$ the set of all (k,l)-quasi-shuffles.

We can easily prove that:

Lemma 1. *Let $k, l \geq 2$. Then, the following sets are in bijections:*

$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{1\}\} \simeq \mathrm{QSh}(k-1, l),$$
$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{k+1\}\} \simeq \mathrm{QSh}(k, l-1),$$
$$\{\sigma \in \mathrm{QSh}(k,l) \,|\, \sigma^{-1}(\{1\}) = \{1, k+1\}\} \simeq \mathrm{QSh}(k-1, l-1).$$

For other cases, we have:

$$\mathrm{QSh}(1,0) = \{\mathrm{Id}_{\{1\}}\} = \mathrm{QSh}(0,1) \text{ and } \mathrm{QSh}(1,1) = \{\mathrm{Id}_{\{1,2\}}, (2,1)\}.$$

Proof. Let k, l be two integers greater or equal to 2.

First case: we denote by $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sigma'(n-1) + 1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k-1, l)$.

Second case: we denote by $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = k+1, \\ \sigma'(n) + 1 & \text{if } n < k+1, \\ \sigma'(n-1) + 1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k, l-1)$.

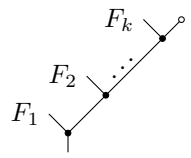
Third case: we denote $\sigma' : [\![1, k+l-1]\!] \to \mathbb{N}$ the unique map such that:

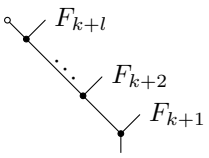
$$\forall n \in [\![1, k+l]\!], \sigma(n) = \begin{cases} 1 & \text{if } n = 1 \text{ or } n = k+1, \\ \sigma'(n-1) + 1 & \text{if } n \leq k, \\ \sigma'(n-2) + 1 & \text{otherwise.} \end{cases}$$

It is left to the reader to check that σ' is an element of $\mathrm{QSh}(k-1, l-1)$.

Hence it establishes the maps from the left to the right in the lemma. Conversely, one remarks in any case that the map $\sigma \mapsto \sigma'$ is invertible. □

Let t, s two trees different from $|$. We look t as a right comb and s as a left comb. In other words, we put:

$t =$ 

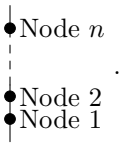
and $s =$ 

,

where for all $i \in [\![1, k+l]\!]$, F_i is a non-empty forest. Here, k represents the number of nodes on the right-most branch of t and l is the number of nodes on the left-most branch of s.

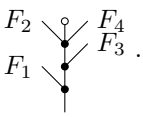
Definition 5. Let t and s be two trees which respective right comb representation and left comb representation are given above. We denote by k (respectively l) the number of nodes on the right-most (respectively left-most) branch of t (respectively s). Let σ be a (k,l)-quasi-shuffle which has for image $[\![1,n]\!]$ with n a positive integer. We denote $\sigma(t,s)$ the tree obtained this way:

(1) We first consider the ladder with n nodes:



(2) For all $i \in [\![1,k]\!]$, we graft F_i as the *left* son at the node $\sigma(i)$.
(3) For all $i \in [\![k+1,k+l]\!]$, we graft F_i as *right* son at the node $\sigma(i)$.

Example 2. Consider the $(2,2)$-quasi-shuffle $\sigma = (1,3,2,3)$.

Let us take $t =$  and $s =$  . Then $\sigma(t,s) =$  .

Theorem 1. *Let t, s be two trees different from $|$ as described above. Then:*

$$t * s = \sum_{\sigma \in \mathrm{QSh}(k,l)} \sigma(t,s).$$

Moreover:

$$t \prec s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(t,s), t \succ s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{k+1\}}} \sigma(t,s),$$

$$t \cdot s = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1,k+1\}}} \sigma(t,s).$$

Surprisingly, $(\mathsf{Sch}, \prec, \cdot, \succ)$ can be endowed with a codendriform coproduct:

Theorem 2. *We define a map $\Delta : \mathbb{K}\,\mathsf{Sch} \to \mathbb{K}\,\mathsf{Sch} \otimes \mathbb{K}\,\mathsf{Sch}$ by:*

$$\Delta(t) = \sum_{c \text{ admissible cut of } t} P^c(t) \otimes R^c(t),$$

where $R^c(t)$ is the component of t containing its root and $P^c(t) = P_1^c(t) * \cdots * P_k^c(t)$, where $P_i^c(t)$ are the cut trees naturally ordered from left to right. Moreover one can write $\Delta = \Delta_\leftarrow + \Delta_\rightarrow$ in such a way so $(\mathsf{Sch}, \prec, \cdot, \succ, \Delta_\leftarrow, \Delta_\rightarrow)$ is a $(3,2)$-dendriform bialgebra detailled in [**?**].

From the Cartier-Quillen-Milnor-Moore theorem, we can understand better the cocommutative part of this Hopf algebra from its primitives. Hence, a natural question is to compute the primitives of Sch. Let us introduce the notations to reach the theorems for this purpose:

Definition 6. Let $(C, \Delta_\leftarrow, \Delta_\rightarrow)$ be a dendriform coalgebra. We define:

$$\mathrm{Prim}_\leftarrow(C) := \{c \in C \mid \tilde{\Delta}_\leftarrow(c) = 0\}, \mathrm{Prim}_\rightarrow(C) := \{c \in C \mid \tilde{\Delta}_\rightarrow(c) = 0\},$$

$$\mathrm{Prim}_{\mathrm{Coass}}(C) := \{c \in C \mid \tilde{\Delta}(c) = 0\}, \mathrm{Prim}_{\mathrm{Codend}}(C) := \{c \in C \mid \tilde{\Delta}_\leftarrow(c) = \tilde{\Delta}_\rightarrow(c) = 0\}.$$

Theorem 3. *For all $n \in \mathbb{N}$, we define:*

$$\theta_n : \left\{ \begin{array}{rcl} \mathrm{Prim}_{\mathrm{Coass}}(\mathsf{Sch}(n)) & \to & \mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(n+1)), \\ a \otimes \curlyvee & \mapsto & a \cdot \curlyvee. \end{array} \right.$$

Then, for all $n \in \mathbb{N}, \theta_n$ is an isomorphism of vector spaces.

From the article [**?**] of M.Ronco, we recall the main result:

Theorem 4. *One can define a map ω using $\succ, \cdot$ and $\prec$ giving rise for all $n \in \mathbb{N}$ to:*

$$\Omega_n : \begin{cases} T(\mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(n))) & \twoheadrightarrow & \mathrm{Prim}_{\mathrm{Coass}}(\mathsf{Sch}(n)), \\ x_1 \otimes \cdots \otimes x_k & \mapsto & \omega(x_1 \otimes \cdots \otimes x_k). \end{cases} \tag{9}$$

This raise to a map Ω defined over Sch such that $\Omega|_{T(\mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(n)))} = \Omega_n$.

As a consequence, we have a recipe to produce the primitives of this algebra by induction as shown in figure 1:

$$\mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(1)) \xrightarrow{\theta_1} \mathrm{Prim}_{\mathrm{Coass}}(\mathsf{Sch}(2)) \xrightarrow{\Omega_1} \mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(2)) \xrightarrow{\theta_1} \mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(3)) \xrightarrow{\Omega_2} \dots$$

FIGURE 1. Diagram of the induction

This figure 1 enables us to compute $\mathrm{Prim}_{\mathrm{Coass}}(\mathsf{Sch})$ by induction over the degree of its elements starting with $\mathrm{Prim}_{\mathrm{Codend}}(\mathsf{Sch}(1)) = \left\{ \right\}$. We will now look at the practical aspects to implement an algorithm on a computer.

3. Our point of view of Schroeder trees and their operations

Our motivation is to compute the process detailed in figure 1. It may give hints to describe some primitives elements or at least make guesses to get an explicit combinatorial formula describing this process. But a question arise from this: "How shall we implement trees to get efficiency and an easy coding ?"

3.1. **Encoding Schroeder trees.**

The idea to represent Schroeder trees into a computer is to add *levels* giving rise to *levelled Schroeder trees*. From this intermediate representation, we can interpret a Schroeder tree as a sequence of integers whose binary code represents the levels of the tree.

3.1.1. *Levelled Schroeder Tree.*

Definition 7. Let $n \in \mathbb{N}$. The set of levelled Schroeder tree is denoted Sch_ℓ and the subset levelled Schroeder trees with n leaves is denoted $\mathsf{Sch}_\ell(n)$.

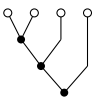
To each levelled Schroeder tree corresponds a unique Schroeder tree with a trivial surjection

$$\pi_n : \mathsf{Sch}_\ell(n) \twoheadrightarrow \mathsf{Sch}(n). \tag{10}$$

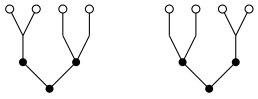
Maps π_0, π_1, π_2 and π_3 are bijective while π_n is not injective for $n \geq 4$.

Definition 8. Two levelled tree t and t' of $\mathsf{Sch}_\ell(n)$ are said *equivalent*, denoted $t \equiv t'$, if it represents the same Schroeder tree, that is, $\pi_n(t) = \pi_n(t')$.

Example 3. Referring to the example 1, except for $i = 7$, the set $\pi_4^{-1}(t_i)$ has only one element. For example $\pi_4^{-1}(t_8)$ consists of the unique levelled tree:



Labelled trees in $\pi_4^{-1}(t_7)$ consists of the following two levelled trees:



We want to code Schroeder trees as levelled Schroeder trees. For this purpose, we need to make a choice of a representative levelled Schroeder tree of each Schroeder tree.

3.1.2. *Our point of view on Schroeder trees.*

Definition 9 (Initial chains)**.** Let $\mathcal{P} = (P, \leq)$ be a lattice with a minimum and a maximum element. Let C be a chain in this poset, we say it is *initial* if $\min(\mathcal{P})$ and $\max(\mathcal{P})$ are in the chain C.

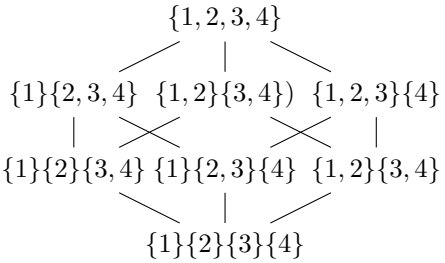
C'est pour l'inclusion non ?

Definition 10. Let $n \in \mathbb{N} \setminus \{0\}$. We define $\mathrm{InitChain}(n)$ the set of strictly increasing *initial chain* of the poset $(\mathrm{Par}(\llbracket 1, n \rrbracket), \leqslant_\pi)$ where $\leqslant_\pi$ is the *refinement order* of partitions. A partition P is smaller than a partition Q, denoted $P \leq_\pi Q$, if any block of Q is the union of some blocks of P.

Example 4. For instance, in the poset $(\mathrm{Par}(\llbracket 1, 4 \rrbracket), \supseteq)$ shown in figure 2, we have the following elements of $\mathrm{InitChain}(4)$:

$$\{1\}\{2\}\{3\}\{4\} \leqslant_\pi \{1,2,3,4\},$$
$$\{1\}\{2\}\{3\}\{4\} \leqslant_\pi \{1,2,3\}\{4\} \leqslant_\pi \{1,2,3,4\}.$$

FIGURE 2. The $\mathcal{P}(\llbracket 1, 4 \rrbracket)$ poset with labels on edges



But as said previously many levelled Schroeder trees give a same Schroeder tree. Hence, for each $n \in \mathbb{N}$ we want to build a section of π_n so that a *unique* levelled Schroeder tree corresponds to a Schroeder tree.

3.1.3. *Our choice of section.* For our use, we introduce the following map:

Proposition 1. *We denote $s : \mathsf{Sch} \to \mathsf{Sch}_\ell$ the unique map such that for any $t \in \mathsf{Sch}(n)$, $s(t)$ is the* unique *levelled Schroeder tree such that for any internal vertices t, its level is strictly greater than all the internal vertices to its right. Then, s is well-defined and this is a section of the map* $\pi \coloneqq \bigoplus_{n=0}^{+\infty} \pi_n$.

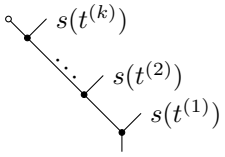
Proof. To prove s is well-defined, we detail a construction $s(t)$ by induction over the number of leaves of the tree of t. Let $t \in \mathsf{Sch}(n)$ with $n \in \mathbb{N}^*$.

Initialization: for $n = 2$, this is obvious as there exists only one representation of as a levelled Schroeder tree.

Heredity: now let us suppose that there exists n such that for any $t \in \mathsf{Sch}(n), s(t)$ is well defined. We consider $t \in \mathsf{Sch}(n+1)$ and let us build $s(t)$. One can see:

$$t = t^{(1)} \vee \cdots \vee t^{(k)}$$

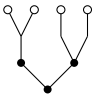
where for any $k \geq 2, i \in [\![1, k]\!], t^{(i)}$ is a Schroeder tree with at most n leaves. Hence, we built $s(t)$ as follows:

$s(t) =$ 

Finally by construction, we have $\pi \circ s = \mathrm{Id}_{\mathsf{Sch}}$ □

Example 5. Consider the tree

The only tree satisfying this definition among its two representatives is:



3.1.4. *The encoding.* For the sequel let us introduce some definitions

Definition 11. Let t be a tree. Let v be a vertex in t. We say that v is an *internal vertex* if it is not a leaf. We will denote $\mathrm{Inn}(t)$ the number of internal vertices of t. Note that for all Schroeder trees this quantity is always non-negative.

Using the section s, we can identify any Schroeder tree with a finite list of binary words thanks to the map defined below

Definition 12. For any $n \in \mathbb{N}$, we introduce a map $\varphi_n : \mathsf{Sch}_\ell(n) \to \mathrm{InitChain}([\![1, n]\!])$ defined for any $t \in \mathsf{Sch}_\ell(n)$ by the *unique* initial chain describing the life time of the angles of t in function of the levels of the tree.

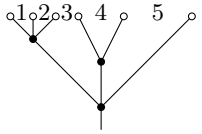
We also define Set_n, the map sending any element of $\mathrm{InitChain}([\![1, n]\!])$ onto a list of length n with 0's and 1's encoding the element of $\mathcal{P}([\![1, n]\!])$ considered. More precisely, for any $X \in \mathcal{P}([\![1, n]\!])$:

$$\mathrm{Set}_n(X) \coloneqq (\mathbb{1}_X(i))_{i \in [\![1,n]\!]}.$$

Finally, we introduce the binary map, denoted B, that sends any sequence with 0's and 1's onto an integer the following way:

$$\forall s \in \{0,1\}^\star, B(s) = \sum_{i=0}^{|s|} s_i \cdot 2^i.$$

Example 6. For instance, reading sequences from top to bottom:

(11) 

$\xrightarrow{\varphi_5}$

$$\begin{matrix} [\![1,5]\!] \\ [\![3,5]\!] \\ \{3\}\ \{5\} \\ \emptyset \end{matrix}$$

$\xrightarrow{\mathrm{Set}_5}$

$$\begin{matrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{matrix}$$

$\xrightarrow{B}$

$$\begin{matrix} 31 \\ 28 \\ 20 \\ 0 \end{matrix}$$

Notation 2. We will call the *code map*, denoted Code, the map $\mathrm{Set} \circ \varphi \circ s : \mathsf{Sch} \to \mathrm{InitChain}$ where:

$$\mathrm{InitChain} \coloneqq \bigoplus_{n=0}^{+\infty} \mathrm{InitChain}(\llbracket 1, n \rrbracket).$$

Thanks to this result we can now encode one Schroeder tree as a list of integers into a computer. Now, we can look at the action of quasi-shuffles on our encoding of trees for an easy computation in the free tridendriform algebra with one generator.

Je mets une partie pour décrire l'image des φ_n que l'on ne conservera peut-être pas.

3.1.5. *Description of the image of Schroeder trees.* We can notice that the image of $\varphi \circ s$, where $\varphi = \bigoplus_{n=0}^{+\infty} \varphi_n$, is strictly included inside $\bigoplus_{n=0}^{+\infty} \mathrm{InitChain}(\llbracket 1, n \rrbracket)$. We give here a description of the image of φ. For this purpose we need to introduce some definitions:

Definition 13. Let $n \in \mathbb{N}$ and $x \in \mathcal{P}(\llbracket 1, n \rrbracket)$. An *interval* in x is a subset of x of the shape $\llbracket i, j \rrbracket$ with $i < j$. We will denote $\mathrm{Con}(x)$ the set of maximal intervals in x.

With this connectivity concept, we have:

$$x = \bigcup_{c \in \mathrm{Con}(x)} c.$$

Remark 1. Let $n \in \mathbb{N}$ et $x \in \mathcal{P}(\llbracket 1, n \rrbracket)$. Note that $\mathrm{Con}(x)$ has a total order induced by the order of the minimum of each block. More precisely, for $(c_1, c_2) \in \mathrm{Con}(x)$:

$$c_1 < c_2 \iff \min(c_1) < \min(c_2).$$

Definition 14. Let $n \in \mathbb{N}$ and consider $C \in \mathrm{InitChain}(\llbracket 1, n \rrbracket)$. Let $x \in C$ which is not an extrema of C. We denote by $x_{\leftarrow}$ the previous element in the chain C. We denote by $x_{\rightarrow}$ the next element in the chain C.

Proposition 2. *We have $C \in \mathrm{Im}(\varphi \circ s)$ if and only if:*

$$\forall x \in C \setminus \{\max(C)\}, \forall c \in \mathrm{Con}(x), \forall c' \in \mathrm{Con}(x), c' > c \implies c' \in x_{\rightarrow}.$$

Proof. Todo □

Est-ce qu'on parle des suites croissantes pour le poids de Hamming ?

3.2. **The tridendriform structure for 0 and 1 sequences.** One usual operation we can perform on words is the concatenation. In our particular case, the tridendriform operations over InitChain are more complicated. In fact, we could multiply two initial chains with partition set of different lengths.

Remark 2. For now on, for any $n \in \mathbb{N}$ we will no longer distinguish element of $\mathrm{InitChain}(\llbracket 1, n \rrbracket)$ and its image by the map Set. We also allow ourselves to mix the notations.

3.2.1. *Preliminary definitions.* In order to describe properly to the computer what we want to do, we have to identify the elements encoding the comb representation of the tree associated to it.

Definition 15 (forest code)**.** Let $n \in \mathbb{N}$. Let $C' \in \mathrm{InitChain}(\llbracket 1, n \rrbracket)$ and put $C = C' \setminus \max(C')$. Any C satisfying this property will be called a *forest code*. The *forest* associated to a forest code is the ordered concatenation of trees we get deleting the root of the tree encoded by C'.

Remark 3. The forest code associated to the empty chain, denoted ε, is $\begin{smallmatrix}\circ\\|\end{smallmatrix}$. Note that if C is a forest code that is not an initial chain, then there exists a unique forest F such that $\mathrm{Code}(T) = C'$ with T is the grafting of all the element of the forest F in this order to a common root. Hence, we extend the definition of the map Code to forests codes in such a way that $\mathrm{Code}(F) = C$.

Example 7. For instance, the forest code Code $\left(\begin{smallmatrix}\circ\\|\end{smallmatrix}\ \begin{smallmatrix}\circ\\|\end{smallmatrix}\ \begin{smallmatrix}\circ\\|\end{smallmatrix}\right)$ is:

$$\begin{array}{ccccc} 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \end{array}$$

Note that Code $\left(\begin{smallmatrix}\circ\\|\end{smallmatrix}\ \begin{smallmatrix}\circ\\|\end{smallmatrix}\ \begin{smallmatrix}\circ\\|\end{smallmatrix}\right) = 1\ 1$.

Definition 16 (left comb decomposition)**.** Let $n \in \mathbb{N}$ and $C \in \mathrm{InitChain}([\![1, n]\!])$. Its *left comb decomposition*, denoted $\mathrm{LCD}\,(C)$, is obtained inductively the following way:

$$\mathrm{LCD}\,(\varepsilon) = \varepsilon$$
$$\mathrm{LCD}\,(C) = (\mathrm{LCD}\,(C_1)\,, F_1),$$

where in the induction we assumed $A \times (B \times C) \approx (A \times B) \times C$, and F_1 and C_1 satisfy the following block equality:

$$C = \left|\begin{array}{ccc} & 1 & \\ \tilde{C}_1 & \vdots & \tilde{F}_1 \\ & 1 & \\ 0\cdots0 & 0 & 0\cdots0 \end{array}\right| \text{ if it is a tree code or } C = \left|\begin{array}{ccc} & 1 & \\ \tilde{C}_1 & \vdots & \tilde{F}_1 \\ & 1 & \end{array}\right| \text{ is a forest code.}$$

such that F_1 and C_1 are respectively $\tilde{F}_1$ and $\tilde{C}_1$ keeping only once any instance of a integer sequence. Moreover, the red line spotted is the *leftmost* column with this property in C.

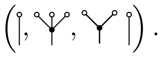
Example 8. We will start from a tree, consider its code, we apply the decomposition detailed above and compare the left comb decomposition of the tree with the forest associated to the sequence of the forest code.

$$s = \quad \longrightarrow \begin{array}{cccccc} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{array} = \mathrm{Code}(s) =: S.$$

Hence:

$$\mathrm{LCD}\,(S) = \left(\mathrm{LCD}\left(\begin{array}{ccc} 1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{array}\right), \begin{array}{cc} 1 & 1 \\ 0 & 1 \end{array}\right)$$
$$= \left(\varepsilon, \begin{array}{cc} 1 & 1 \\ 0 & 0 \end{array}, \begin{array}{cc} 1 & 1 \\ 0 & 1 \end{array}\right).$$

Associating to it the 3-uplet of the forests code by inverting Code and applying it component-wise, we have:



We have recovered the left comb decomposition of s after forgetting the leftmost .

Definition 17 (right comb decomposition). Let $n \in \mathbb{N}$ and $C \in \mathrm{InitChain}([\![1,n]\!])$. Its *right comb decomposition*, denoted $\mathrm{RCD}\,(C)$, is obtained inductively the following way:

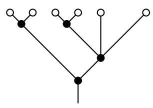
$$\mathrm{RCD}\,(\varepsilon) = \varepsilon$$
$$\mathrm{RCD}\,(C) = (F_1, \mathrm{RCD}\,(C_1)),$$

where in the induction we assumed $A \times (B \times C) \approx (A \times B) \times C$, and F_1 and C_1 satisfy the following block equality:

$$C = \left| \begin{array}{c|c|c} \tilde{F_1} & \begin{array}{c} 1 \\ \vdots \\ 1 \end{array} & \tilde{C_1} \\ 0\cdots 0 & 0 & 0\cdots 0 \end{array} \right| \text{ if it is a tree code or } C = \left| \begin{array}{c|c|c} \tilde{F_1} & \begin{array}{c} 1 \\ \vdots \\ 1 \end{array} & \tilde{C_1} \end{array} \right| \text{ if it is a forest code.}$$

such that F_1 and C_1 are respectively $\tilde{F_1}$ and $\tilde{C_1}$ keeping only once any instance of a integer sequence. Moreover, the red line spotted is the *rightmost* column in C ending with a 1 or with a unique 0 if the last line is full of zeros.

Example 9. We will start from a tree, consider its code, then apply the decomposition detailed above and compare the right comb decomposition of the tree with the forest associated to the sequence of forest code.

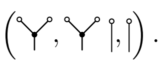
$t =$ 

$$\longrightarrow \begin{array}{ccccc} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{array} = \mathrm{Code}(t) =: T$$

Hence:

$$\mathrm{RCD}\,(T) = \left(\begin{array}{c} 1 \\ 0 \end{array}, \mathrm{RCD}\left(\begin{array}{ccc} 1 & 1 & 1 \\ 0 & 1 & 1 \end{array} \right) \right)$$
$$= \left(\begin{array}{c} 1 \\ 0 \end{array}, \begin{array}{cc} 1 & 1 \\ 0 & 1 \end{array}, \varepsilon \right)$$

Associating to it the 3-uplet of the forests code, we have:



We have recovered the comb decomposition of t after forgetting the rightmost .

Definition 18. For any Schroeder tree t, we define respectively the *right tree length* and the *left tree length* by:

$$\mathrm{RTL}(t) := |\,\mathrm{RCD}\,(t)\,| - 1 \text{ and } \mathrm{LTL}(t) := |\,\mathrm{LCD}\,(t)\,| - 1.$$

Now, we have an analogous of the comb decomposition for our sequences. We can now describe the action of a quasi-shuffle on these objects.

3.2.2. *The tridendriform structure for initial chains.* By analogy with Schroeder trees, we put:

Definition 19. Let $T \in \mathrm{InitChain}([\![1,n]\!])$ and $S \in \mathrm{InitChain}([\![1,m]\!])$. Let us denote $k = \mathrm{RTL}(T)$ and $l = \mathrm{LTL}(S)$. Consider $\sigma \in \mathrm{QSh}(k,l)$ which has for image $[\![1,r]\!]$. We denote by $\sigma(T,S)$ the element of $\mathrm{InitChain}([\![1,m+n]\!])$ defined by:

(1) on the first m integers, copy all the k forests codes of $\mathrm{RCD}\,(T)$ (excepted to the rightmost one $\mathring{|}$) separated by a column of ones on their right;
(2) for h from r to 1 with a -1 step:
   Case 1: if $\sigma^{-1}(\{h\}) = \{i\}, i \in [\![1,k]\!]$, write 0's below the code of F_i and on its adjacent right column.
   Case 2: if $\sigma^{-1}(\{h\}) = \{j\}, j \in [\![k+1,k+l]\!]$, include the code of F_j at its appropriate place shifted by n with a ones column on its left write 0's below the code of F_j and on its adjacent left column. Write 0's below the code of F_j and on its adjacent left column.
   Case 3: if $\sigma^{-1}(\{h\}) = \{i,j\}, (i,j) \in [\![1,k]\!] \times [\![k+1,k+l]\!]$, include the code of F_j at its appropriate place shifted by n with a ones column on its left. Write 0's below the code of F_j and on its adjacent left column. Then, write 0's below the code of F_j and on its adjacent left column *and* below the code of F_i and on its adjacent right column.

The element obtained from this operation is denoted $\sigma(T,S)$.

Remark 4. For this operation, one should remind the ending positions of each forest code.

Example 10. Consider the $(2,2)$-quasi-shuffle $\sigma = (1,3,2,3)$. Let us take $t = f_1\ f_2$ and $s = f_3\ f_4$. Then $\sigma(t,s) = \begin{matrix} f_1 \\ f_2 \end{matrix}\ f_3\ f_4$. Then, let us denote for all $i \in [\![1,4]\!], \mathrm{Code}(f_i) = F_i$. Hence, the code associated to t and s respectively denoted T and S are of the shape:

$$T = \begin{array}{cccccc}
1 & \cdots & 1 & 1 & \cdots & 1 \\
 & & 1 & 1 & \cdots & 1 \\
 & F_1 & \vdots & 1 & \cdots & 1 \\
 & & 1 & 1 & \cdots & 1 \\
\vdots & \vdots & 1 & & & 1 \\
\vdots & \vdots & 1 & & F_2 & \vdots \\
\vdots & \vdots & 1 & & & 1 \\
\vdots & \vdots & 1 & 0 & \cdots & 0 \\
0 & \cdots & 0 & 0 & \cdots & 0
\end{array}
\quad \text{and} \quad S = \begin{array}{cccccc}
1 & \cdots & 1 & 1 & \cdots & 1 \\
1 & & & 1 & \cdots & 1 \\
\vdots & & F_4 & & 1 & \cdots & 1 \\
1 & & & 1 & \cdots & 1 \\
0 & \cdots & 0 & 1 & \cdots & 1 \\
0 & \cdots & 0 & 1 & & \\
0 & \cdots & 0 & \vdots & F_3 & \\
0 & \cdots & 0 & 1 & & \\
0 & \cdots & 0 & 0 & \cdots & 0
\end{array}$$

Then, $\sigma(T,S)$ is the code of the tree $\sigma(t,s)$ given by:

$$
\begin{array}{cccccccccccc}
1 & \cdots & 1 & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & 1 \\
 & & 1 & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & 1 \\
 F_1 & & \vdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \cdots & \vdots \\
 & & 1 & 1 & \cdots & 1 & 1 & \cdots & \cdots & \cdots & \cdots & 1 \\
\vdots & \vdots & \vdots & & & \vdots & \vdots & \cdots & \cdots & \cdots & \cdots & \vdots \\
\vdots & \vdots & \vdots & & F_2 & & 1 & 1 & \cdots & \cdots & \cdots & \cdots & 1 \\
\vdots & \vdots & 1 & & & & 1 & 1 & \cdots & \cdots & 1 & \cdots & 1 \\
\vdots & \vdots & 1 & \vdots & \vdots & 1 & 1 & & & & \vdots & \cdots & \vdots \\
\vdots & \vdots & 1 & \vdots & \vdots & \vdots & \vdots & & F_4 & & \vdots & \cdots & \vdots \\
\vdots & \vdots & 1 & \vdots & \vdots & 1 & 1 & & & & 1 & \cdots & 1 \\
\vdots & \vdots & 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 1 & \cdots & 1 \\
\vdots & \vdots & 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 1 & & \\
\vdots & \vdots & 1 & 0 & \cdots & 0 & 0 & \cdots & \vdots & \vdots & F_3 & \\
\vdots & \vdots & 1 & 0 & \cdots & 0 & 0 & \cdots & 0 & 1 & & \\
\vdots & \vdots & 1 & 0 & \cdots & 0 & 0 & \cdots & \cdots & 0 & \cdots & 0 \\
0 & \cdots & \cdots & \cdots & \cdots & 0 & 0 & \cdots & \cdots & 0 & \cdots & 0
\end{array}
$$

With this definition, we are now able to define three operators in Im(Code)

Definition 20. Let T and S be two elements in Im(Code) such that $k = |\operatorname{RTL}(T)|$ and $l = |\operatorname{LTL}(S)|$. We define:

$$T \prec S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(T,S), \qquad T \succ S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1\}}} \sigma(T,S),$$

$$T \cdot S = \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}(\{1\}) = \{1,k+1\}}} \sigma(T,S).$$

In the sequel, we show that this structure is a tridendriform structure on Im(Code) showing it is isomorphic to the free tridendriform algebra of Schroeder trees. Hence it gives us a mathematical background to implement programs in section 4.

Theorem 5. *Let t, s be two Schroeder trees with shapes given in equation* (1)*. Hence, k is the right comb length of t and l the left comb length of s. Let $\sigma \in \mathrm{QSh}(k,l)$. Then:*

$$\mathrm{Code}(\sigma(t,s)) = \sigma(\mathrm{Code}(t), \mathrm{Code}(s)).$$

Proof. In order to prove this theorem, we will proceed by induction over the sum $\max(k,l)$. If $l=0$ or $k=0$, this means that $s=\,\vert\,$ or $t=\,\vert\,$.

Initialization: if $\max(k,l)=1$ and $k+l=1$, then the identity shuffle is acting on this pair of tree and one of them is $\vert$. By definition 19, we get back the tree code of the non-unit element. Otherwise, if $k+l=2$ hence $\sigma=(1,1)$, then:

$$t=\ f_1\ \curlyvee,\ s=\ \curlyvee\ f_2\ \text{ and } \sigma(t,s)=\ f_1\ \curlyvee\ f_2$$

Denoting $\mathrm{Code}(f_1)=F_1$ and $\mathrm{Code}(f_2)=F_2$, their respective codes are:

$$T=\begin{array}{ccc} 1&\cdots&1\\ &&1\\ &F_1&\vdots\\ &&1\\ 0&\cdots&0 \end{array},\ S=\begin{array}{ccc} 1&\cdots&1\\ 1&&\\ 1&F_2&\\ 0&\cdots&0 \end{array}\ \text{ and } \sigma(T,S)=\begin{array}{cccccccc} 1&\cdots&1&1&1&1&\cdots&1\\ &&&1&1&1&\cdots&1\\ &F_1&&\vdots&\vdots&\vdots&\vdots&\vdots\\ &&&1&1&1&\cdots&1\\ 0&\cdots&0&1&1&&&\\ 0&\cdots&0&\vdots&\vdots&&F_2&\\ 0&\cdots&0&1&1&&&\\ 0&\cdots&0&0&0&0&\cdots&0 \end{array}$$

Note that the code of $\sigma(T,S)$ has for associated tree $\sigma(t,s)$.

Heredity: Suppose that there exists $r\geq 1\in\mathbb{N}^*$ such that for all couple (k,l) with $\max(k,l)\leq r$, we have:

$$\sigma(\mathrm{Code}(t),\mathrm{Code}(s))=\mathrm{Code}(\sigma(t,s)).$$

Let us prove that this property is true for $r+1$. Let t,s be two trees with right comb length k and left comb length l, let $\sigma\in\mathrm{Sh}(k,l)$ such that $\max(k,l)=r+1$. Note that if $k=0$ or $l=0$, the result is obvious by definition 20. There are three possible cases left, in those cases, we use the notations of the proof of lemma 1:

First case: $\sigma^{-1}(\{1\})=\{1\}$: Then:

$$\mathrm{Code}(\sigma(t,s))=\mathrm{Code}\left(\ f_1\ \curlyvee\ \sigma'(t',s)\ \right)\ \text{where } t'=\ f_1\ \curlyvee\ t'\ ,$$

$$=\begin{array}{ccccccc} 1&\cdots&1&1&\cdots&1\\ &&1&1&\cdots&1\\ F_1&&\vdots&1&\cdots&1\\ &&1&1&\cdots&1\\ \vdots&\vdots&1&&&\\ \vdots&\vdots&\vdots&\multicolumn{3}{c}{\mathrm{Code}(\sigma'(t',s))}\\ \vdots&\vdots&1&&&\\ 0&\cdots&0&0&\cdots&0 \end{array}$$

Applying the induction hypothesis, we have:

$$\mathrm{Code}(\sigma(t,s)) = \begin{array}{ccccccc} 1 & \cdots & 1 & 1 & \cdots & & 1 \\ & & & 1 & 1 & \cdots & 1 \\ & F_1 & & \vdots & 1 & \cdots & 1 \\ & & & 1 & 1 & \cdots & 1 \\ \vdots & & \vdots & 1 & & & \\ \vdots & & \vdots & \vdots & \sigma'(\mathrm{Code}(t'),\mathrm{Code}(s)) & & \\ \vdots & & \vdots & 1 & & & \\ 0 & \cdots & 0 & 0 & \cdots & & 0 \end{array}$$

$$= \sigma(\mathrm{Code}(t),\mathrm{Code}(s)),$$

by definition of the action of σ over a pair of forests codes.

Second case: $\sigma^{-1}(\{1\}) = \{k+1\}$ is similar to the first one. So it is left to the reader.

Third case: $\sigma^{-1}(\{1\}) = \{1, k+1\}$. Then:

$$\mathrm{Code}(\sigma(t,s)) = \mathrm{Code}\left(\begin{array}{c} \sigma'(t',s') \\ f_1 \quad\quad f_{k+1} \end{array} \right) \text{ where } s = \begin{array}{c} s' \quad f_{k+1} \end{array}$$

$$= \begin{array}{ccccccccccc} 1 & \cdots & 1 & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ & & & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots \\ & F_1 & & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ & & & 1 & 1 & \cdots & 1 & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & & & & \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \mathrm{Code}(\sigma'(t',s')) & & & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & & & & 1 & 1 & \cdots & 1 \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & & & \\ \vdots & \cdots & \vdots & \vdots & 0 & \cdots & \vdots & \vdots & & F_{k+1} & \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & & & \\ 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \end{array}$$

Applying the induction hypothesis, we have $\mathrm{Code}(\sigma'(t',s')) = \sigma(\mathrm{Code}(t'),\mathrm{Code}(s'))$. Then applying the construction case number 3 in definition 19, one has:

$$\mathrm{Code}(\sigma(t,s)) = \sigma(\mathrm{Code}(t),\mathrm{Code}(s)).$$

Hence by the induction principle, we have proved the theorem. □

Finally, the map Code is also linear hence:

Corollary 1. *The* Code *map is an isomorphism of tridendriform algebras into its image.*

Proof. To prove this corollary, one just needs to show that Code is a tridendriform morphism from $(\mathsf{Sch}, \prec, \succ, \cdot)$ into $(\mathrm{Im}(\mathrm{Code}), \prec, \succ, \cdot)$. Let $t, s \in \mathsf{Sch}$ such t has right comb length k and s has left comb length l. We show Code is a morphism for the middle product:

$$\begin{aligned}\mathrm{Code}(t \cdot s) &= \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}\{1\} = \{1, k+1\}}} \mathrm{Code}(\sigma(t, s)) \\ &= \sum_{\substack{\sigma \in \mathrm{QSh}(k,l) \\ \sigma^{-1}\{1\} = \{1, k+1\}}} \sigma(\mathrm{Code}(t), \mathrm{Code}(s)) \\ &= \mathrm{Code}(t) \cdot \mathrm{Code}(s).\end{aligned}$$

The other cases are similar. □

Now that we are sure the two descriptions are isomorphic, we can get into coding.

4. Implementing the problem

A seen in last corollary, we will represent Schroeder trees as an element of Im(Code). As a consequence, we have to build enumerations and structures needed to perform computations:

- enumeration of quasi-shuffles;
- class describing Schroeder trees as sequences of 0's and 1's;
- computation of complexity and comparison with the naive algorithm.

But first, let us look at the efficiency of a naive approach.

4.1. The naive implementation.

One naive way to implement this is to use a recursive description given by [**?**] using the grafting operator:

$$\begin{aligned} t \prec s &= t^{(1)} \vee \cdots \vee \left(t^{(k)} * s\right), \\ t \cdot s &= t^{(1)} \vee \cdots \vee \left(t^{(k)} * s^{(1)}\right) \vee \cdots \vee s^{(l)}, \\ t \succ s &= \left(t * s^{(1)}\right) \vee \cdots \vee s^{(l)}, \end{aligned}$$

where $t = t^{(1)} \vee \cdots \vee t^{(k)}$ and $s = s^{(1)} \vee \cdots \vee s^{(l)}$, such that $| * t = t = t * |$ for all t.

The intuitive algorithm associated to this approach is:

```
def Prod_rec(t1,t2):
L1 = Get_subtrees(t1)
L2 = Get_subtrees(t2)
Left_prod = Graft(L1[0:-1], Prod_rec(L1[-1], L2))
Right_prod = Graft( Prod_rec(L1, L2[-1]), L2[0:-1])
Middle_prod = Graft(L1[0:-1], Prod_rec(L1[-1],L2[0]) , L2[0:-1])
return (Left_prod + Right_prod + Middle_prod)
```

Denote $T(n, m)$ the complexity this algorithm where n and m are respectively the number of leaves of t_1 and t_2. Hence, denoting the depth p_1 of t_1 and p_2 of t_2, one has:

$$T(n, m) = T\left(\frac{n}{2}, m\right) + T\left(n, \frac{m}{2}\right) + T\left(\frac{n}{2}, \frac{m}{2}\right) + np_1 + mp_2.$$

Actually, with our encoding, `Get_subtrees` needs to read each n column on p_1 lines to find subtrees. Let us approximate the complexity where t_1 and t_2 are both binary trees, have the

same number of leaves and at each step the decomposition splits the leaves in two sets of same cardinality. Then denoting $\tilde{T}(n+m) = T(n,m)$, one has:

$$2\tilde{T}\left(\frac{3}{4}n\right) + (n-1)\ln_2(n) \leq \tilde{T}(n) \leq 3\tilde{T}\left(\frac{3}{4}n\right) + (n-1)\ln_2(n).$$

Hence applying the master theorem to compute bounds gives $\left(\ln_{\frac{4}{3}}(2) \approx 2.40, \ln_{\frac{4}{3}}(3) \approx 3.80\right)$:

$$O(n^{2.40}) \leq \tilde{T}(n) \leq O(n^{3.80}).$$

4.2. **Quasi-shuffle enumeration.** In the code available on GitHub [**?**], one can find a procedure `Quasishuffle` that returns the list of all quasi-shuffle of a given type (n,m). For this, we use the result from lemma 1.